Como crear controles de datos

Repositorio de controles por tipo de modelo

VIRF Agent - Validacion de Modelos. Guia para llenar y ampliar el repositorio inputs/repositorio_controles/ cuando aparece un nuevo tipo de modelo (VaR, LaR, ALM, conductuales, u otro).

1. Que es y para que sirve

El repositorio es una biblioteca de controles de calidad de datos escritos como FUNCIONES PYTHON, organizados por tipo de modelo. Cuando eliges el tipo de modelo en Configuracion, el notebook entregable (.ipynb) toma los controles de la carpeta de ese dominio, embebe su codigo real en cada celda, lo ejecuta y entrega un cuadro de 8 columnas. Asi el notebook es ESPECIALIZADO por modelo, no generico.

2. Estructura de carpetas

Una carpeta por dominio (igual que el RAG). El tipo de modelo elige la carpeta:

```
inputs/repositorio_controles/
  var/            kupiec.py  christoffersen.py  var_negativos.py  ...
  lar/            horizonte_supervivencia.py
  alm/            gap_reprecios.py
  conductuales/   cpr_prepago.py  decaimiento_nmd.py
  shared/         (se anaden a CUALQUIER dominio)
```

Que carpeta usa cada tipo de modelo

- Tipo 'VaR' -> carpeta var/
- Tipo 'LaR' -> carpeta lar/ (alias: 'Liquidity at Risk', 'liquidez')
- Tipo 'ALM' -> carpeta alm/
- Tipo 'conductuales' -> carpeta conductuales/
- Cualquier otro tipo -> carpeta con ese nombre en minuscula

3. Anadir un NUEVO tipo de modelo

Si en Configuracion aparece un tipo de modelo que aun no tiene controles:

- 1) Crea una carpeta con el nombre del dominio en minuscula, p.ej. inputs/repositorio_controles/ifrs9/
- 2) Coloca dentro uno o varios archivos .py (un control por archivo).
- 3) Vuelve a descargar el Notebook entregable: tus controles ya aparecen como celdas.

No hay que tocar codigo del sistema ni reiniciar: el repositorio se lee al generar el notebook.

4. Anatomia de un control (.py)

Cada archivo define DOS cosas: un diccionario META (los metadatos que llenan el cuadro) y una funcion control(...) que hace el calculo.

META - los metadatos del cuadro

```
META = {
    "descripcion": "...",   # -> columna 'Control Esperado'
    "dimension":   "...",   # -> columna 'Dimension'
    "evidencia":   "...",   # -> columna 'Control o Evidencia requerida'
    "referencia":  "...",   # -> columna 'Referencia en el entregable'
    "orden":       10,       # (opcional) orden en el notebook
    "params":      {          # (opcional) columnas/umbrales EDITABLES
        "campo_var": "var_99", "nivel_confianza": 0.99,
    },
}
```

control(df, datasets, params) - el calculo

- df: el dataset principal (DataFrame de pandas), o None si no hay datos.
- datasets: diccionario {nombre: DataFrame} con todos los datasets cargados.
- params: una copia de META['params'] (el validador puede editarlos en el notebook).

La funcion DEBE devolver un diccionario con estos tres campos:

```
return {
    "pct":         99.4,          # % de cumplimiento (numero) o None
    "decision":    "Cumple",      # ver valores validos abajo
    "diagnostico": "3 excep. en 500 obs...",  # texto corto para el cuadro
}
```

Valores validos de 'decision'

- Cumple -> control conforme
- Observado -> conforme con salvedad / zona amarilla
- No cumple -> incumplimiento
- No evaluable -> faltan datos/columnas para ejecutarlo
- Revision manual -> requiere criterio del validador

5. Plantilla lista para copiar

Copia este esqueleto en un nuevo .py dentro de la carpeta del dominio y ajusta META y el cuerpo de control(). Es un control real y minimo (valida un rango):

```
"""Control <DOMINIO>: <que verifica en una linea>."""

META = {
    "descripcion": "El campo X debe estar dentro del rango esperado.",
    "dimension":   "Validez",
    "evidencia":   "Serie del campo X por fecha.",
    "referencia":  "SR 11-7 / norma aplicable",
    "orden":       10,
    "params":      {"dataset": "", "campo": "x", "min": 0.0, "max": 1.0},
}

def control(df, datasets, params):
    import pandas as pd
    col = params.get("campo", "x")
    if df is None or col not in getattr(df, "columns", []):
        return {"pct": None, "decision": "No evaluable",
                "diagnostico": f"falta la columna {col!r}"}
    v = pd.to_numeric(df[col], errors="coerce")
    n = int(v.notna().sum())
    if n == 0:
        return {"pct": None, "decision": "No evaluable", "diagnostico": "sin datos"}
    lo, hi = params.get("min"), params.get("max")
    fuera = int(((v < lo) | (v > hi)).sum())
    pct = round(100 * (1 - fuera / n), 2)
    decision = "Cumple" if fuera == 0 else "No cumple"
    return {"pct": pct, "decision": decision,
            "diagnostico": f"{fuera} de {n} valores fuera de [{lo}, {hi}]"}
```

6. Que genera el notebook

Por cada control, una celda que embebe su codigo, lo ejecuta e imprime:

```
>>>> C01: El campo X debe estar dentro del rango esperado.
Control   | Resultado   | % cumplimiento | Diagnostico
C01       | Cumple      | 100.00%        | 0 de 500 valores fuera de [0.0, 1.0]
```

Y una celda final con el cuadro consolidado de 8 columnas:

Control - Dimension - Control Esperado - % De cumplimiento del data set - Diagnostico - Decision - Control o Evidencia requerida - Referencia en el entregable.

7. Checklist antes de guardar el control

- El archivo esta en la carpeta del dominio correcto.
- Define META (con descripcion, dimension, evidencia, referencia).
- Define control(df, datasets, params) y devuelve {pct, decision, diagnostico}.
- Usa params para los nombres de columna y umbrales (editables en el notebook).
- Si faltan datos, devuelve decision='No evaluable' (no lanza excepcion).
- Vuelve a descargar el Notebook entregable y revisa que tu control aparezca.